



南開大學

《SIMD 编程实验》

矩阵 SVD 分解算子 SIMD 并行优化编程实验

姓名: 尚文轩

学号: 2412456

专业: 计算机科学与技术

2026 年 5 月 8 日

目录

摘要	2
1 问题描述	2
1.1 SVD 分解背景	2
1.2 实验目标与核心问题	2
2 SVD 算法流程与可并行化分析	3
2.1 两阶段 SVD 求解框架	3
2.2 Householder 变换的数学结构	3
2.3 GKH 迭代中的 Givens 旋转	4
2.4 并行化点筛选	4
3 SIMD 优化设计与实现	4
3.1 AArch64 NEON 指令集适配	4
3.2 Householder 连续点积与 scaled-add	4
3.3 Householder 中访存方向重排	5
3.4 GKH 行旋转指针化与循环展开	6
3.5 未实现并行点的说明	7
4 实验环境与测试规范	7
4.1 实验环境	7
4.2 测试规范与数据记录	8
5 实验结果与性能分析	8
5.1 正确性验证	8
5.2 递进式消融实验	8
5.3 编译优化等级对比	9
5.4 perf 端到端观察	9
5.5 汇编级分析	10
6 进阶探索与反思	11
6.1 手写 SIMD 与编译器自动优化	11
6.2 关于收敛扫描与活动块并行的反思	12
6.3 AI 辅助实验设计与验证反思	12
7 结论	13
A 代码开放获取	13
B 实验结果文件组织	13
C AI 辅助编程对话摘录	14
C.1 架构选择	14
C.2 GKH 显式 NEON 性能不稳定	14
C.3 Householder 访存重排	14
C.4 perf 与汇编分析	14

摘要

奇异值分解 (Singular Value Decomposition, SVD) 是科学计算、机器学习和高性能矩阵算子优化中的基础算法。经典稳定实现通常先通过 Householder 变换将一般矩阵化为上二对角矩阵, 再对二对角矩阵执行 Golub–Kahan 型隐式迭代求解奇异值 [1, 2, 3]。本实验基于课程提供的 SVD 框架, 在 OpenEuler AArch64 服务器环境下, 围绕上二对角化阶段的 Householder 变换和 GKH 迭代阶段的 Givens 旋转进行 SIMD 优化。

实验的核心思想不是机械地把所有循环替换为 SIMD Intrinsics, 而是先分析算子的数学结构、访存连续性和控制依赖, 再选择适合 SIMD 的路径优化。在 Householder 阶段, 本实验将原始代码中按列累加的矩阵–向量乘法重排为按行连续访问, 并使用 ARM NEON 辅助函数优化连续点积与连续 scaled-add 运算, 使核心 rank-1 更新更符合 128-bit NEON 双精度向量执行模型 [4]。在 GKH 阶段, 本实验对 `apply_left_rows` 采用行指针访问与 4 路循环展开, 减少 `Matrix::at()` 的重复下标计算与循环控制开销; 同时保留显式 NEON 尝试版本作为对照, 分析手工 SIMD 在小粒度、不连续访存场景下可能出现的性能陷阱。

实验通过 baseline、GKH-only、bidiag-only 与 final 四类消融实验进行验证。代表性 O2 结果显示, final 版本在保持全部测试样例正确通过的前提下, 将上二对角化总耗时从 10342.5 ms 降至 3651.82 ms, GKH 迭代耗时从 41470.2 ms 降至 30509.9 ms, 总耗时从 51812.7 ms 降至 34161.72 ms, 整体加速比约为 1.52 倍。编译优化等级对比、端到端 perf 观察与汇编级分析进一步表明: Householder 阶段的收益主要来自访存重排和 NEON 向量化; GKH 阶段的收益主要来自指针化和循环展开, 而不是盲目显式向量化。

1 问题描述

1.1 SVD 分解背景

奇异值分解是线性代数中最重要的矩阵分解之一。对于任意实矩阵 $A \in \mathbb{R}^{m \times n}$, 本实验框架设定 $m \geq n$, 其 SVD 形式为:

$$A = U \Sigma V^T, \quad (1)$$

其中 $U \in \mathbb{R}^{m \times m}$ 与 $V \in \mathbb{R}^{n \times n}$ 是正交矩阵, $\Sigma \in \mathbb{R}^{m \times n}$ 是对角矩阵, 对角线元素 $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_n \geq 0$ 称为奇异值。

SVD 相比特征值分解更具普适性, 能够处理非方阵和秩亏损矩阵, 因此被广泛用于 PCA、低秩近似、图像压缩、噪声抑制、推荐系统以及深度学习中的权重分析。直接通过 $A^T A$ 求特征值虽然概念简单, 但会放大条件数, 数值稳定性较差。因此, 实际数值计算中更常采用 Golub–Kahan 思路: 先用 Householder 变换将一般矩阵化为上二对角矩阵, 再在二对角矩阵上执行隐式 QR/GKH 迭代 [1, 2]。

课程提供的 SVD 框架也遵循这一两阶段流程: `bidiagonalization.cpp` 负责将矩阵 A 化为上二对角矩阵 B , 并累计正交矩阵 U 、 V ; `gkh.cpp` 在上二对角矩阵上进行带 Wilkinson shift 的 GKH 迭代和 bulge chasing, 最终使 B 收敛为对角矩阵。

1.2 实验目标与核心问题

本次实验的主题是 SVD 算子的 SIMD 并行优化。助教讲解中提到的潜在优化方向包括: 矩阵乘法/矩阵向量计算优化、GKH 迭代中的收敛扫描、活动块分裂后的并行处理等。结合 SIMD 实验

的重点，本实验将优化目标具体化为以下问题：

- 1) 在 Householder 上二对角化中，如何把矩阵-向量乘法和 rank-1 更新映射到连续 SIMD 数据通路？
- 2) 在 GKH 迭代中，哪些 Givens 旋转操作适合优化，哪些操作不适合强行显式 SIMD？
- 3) 如何在保证 $A = UBV^T$ 不变量、正交性和奇异值排序正确的前提下减少运行时间？
- 4) 如何通过消融实验、编译优化等级对比和汇编级分析解释优化为什么有效？

因此，本实验不仅关注最终加速比，也关注不同优化策略的适用条件及其性能差异。

2 SVD 算法流程与可并行化分析

2.1 两阶段 SVD 求解框架

本框架的 SVD 求解可以概括为两个阶段：

$$A \xrightarrow{\text{Householder}} B \xrightarrow{\text{GKH iteration}} \Sigma. \quad (2)$$

第一阶段使用左右 Householder 变换将一般矩阵 A 化为上二对角矩阵 B 。所谓上二对角矩阵，是指除主对角线和上次对角线外，其余元素理论上均为 0。该阶段同时更新 U, V ，保持：

$$A = UBV^T. \quad (3)$$

第二阶段在 B 上执行 GKH 迭代。由于 B 已经是上二对角形式，迭代可以只通过一系列 Givens 旋转维护稀疏结构，并将上次对角线逐渐压缩为 0。最终 B 被整理为非负且降序的对角矩阵，其对角线元素即奇异值。

2.2 Householder 变换的数学结构

以第 k 步左 Householder 变换为例，设从第 k 列下方提取的子向量为 x ，构造 Householder 向量 v ，定义：

$$H = I - \beta vv^T, \quad \beta = \frac{2}{v^T v}. \quad (4)$$

左乘更新为：

$$B_{sub}^{new} = HB_{sub} = B_{sub} - \beta v(v^T B_{sub}). \quad (5)$$

代码上可拆成两个计算阶段：

$$w = v^T B_{sub}, \quad (6)$$

$$B_{sub} \leftarrow B_{sub} - \beta vw^T. \quad (7)$$

这一拆分非常重要。 $w = v^T B_{sub}$ 是向量-矩阵乘法； $B_{sub} \leftarrow B_{sub} - \beta vw^T$ 是 rank-1 更新。二者都包含大量重复的乘加操作，且如果能沿矩阵行方向访问，就能形成连续内存流，适合 SIMD。

右 Householder 变换同理，只是作用方向变为：

$$B_{sub}^{new} = B_{sub}H = B_{sub} - \beta(B_{sub}v)v^T. \quad (8)$$

2.3 GKH 迭代中的 Givens 旋转

完成上二对角化后, GKH 迭代通过 Wilkinson 偏移和 bulge chasing 将上次对角线元素逐渐消去。该过程频繁调用 Givens 旋转。对两行 r_0, r_1 左乘旋转时, 核心线性组合为:

$$\begin{cases} a' = ca + sb, \\ b' = -sa + cb. \end{cases} \quad (9)$$

对两列 c_0, c_1 右乘旋转时, 计算形式类似, 但访存模式不同。由于 **Matrix** 按行主序存储, 行旋转访问两整行, 内存连续性较好; 列旋转需要逐行访问相邻两列, 虽然每行内两个元素相邻, 但整体跨行跳转, 局部性弱于行旋转。因此, 二者不应采用完全相同的优化策略。

2.4 并行化点筛选

结合算法结构和实验主题, 本实验对潜在并行化点进行筛选:

- **Householder 矩阵-向量计算**: 计算密度高、循环长、可重排为连续访存, 是 SIMD 优化重点。
- **GKH 行旋转**: 行主序下连续性较好, 适合指针化和循环展开; 显式 NEON 需谨慎。
- **GKH 列旋转**: 访存局部性较差, 强行 SIMD 收益不稳定, 最终保留原始实现。
- **收敛扫描和活动块划分**: 具有并行潜力, 但分支多、计算量小, 更适合多线程任务调度, 不是本次 SIMD 优化重点。

3 SIMD 优化设计与实现

3.1 AArch64 NEON 指令集适配

实验平台为 OpenEuler AArch64 环境, 因此不能使用 x86 AVX 指令集, 而应使用 ARM NEON/ASIMD。NEON 向量寄存器宽度为 128 bit。本实验使用 double 进行数值计算, 每个 double 占 64 bit, 因此一个 NEON 向量寄存器一次处理两个 double 元素。代码中使用 `float64x2_t` 表示双精度向量, 并使用 `vld1q_f64`、`vst1q_f64`、`vdupq_n_f64`、`vaddq_f64`、`vmulq_f64` 等接口 [4]。

3.2 Householder 连续点积与 scaled-add

为了降低修改风险并复用 SIMD 逻辑, 本实验首先封装连续数组点积和连续 scaled-add 两个基础算子。核心代码如代码 1 所示。

Listing 1: Householder 优化中的连续点积与 scaled-add 辅助函数

```
1  #if defined(__aarch64__)
2  #include <arm_neon.h>
3  #endif
4
5  static double dot_product_contiguous(const double *a, const double *b, int n)
6  {
7      double sum = 0.0;
8  #if defined(__aarch64__)
```

```

9     float64x2_t vsum = vdupq_n_f64(0.0);
10    int i = 0;
11    for (; i + 1 < n; i += 2)
12    {
13        float64x2_t va = vld1q_f64(a + i);
14        float64x2_t vb = vld1q_f64(b + i);
15        vsum = vaddq_f64(vsum, vmulq_f64(va, vb));
16    }
17    double tmp[2];
18    vst1q_f64(tmp, vsum);
19    sum = tmp[0] + tmp[1];
20    for (; i < n; ++i) sum += a[i] * b[i];
21 #else
22     for (int i = 0; i < n; ++i) sum += a[i] * b[i];
23 #endif
24     return sum;
25 }
26
27 static void add_scaled_contiguous(double *dst, const double *src,
28                                   double alpha, int n)
29 {
30     #if defined(__aarch64__)
31         float64x2_t valpha = vdupq_n_f64(alpha);
32         int i = 0;
33         for (; i + 1 < n; i += 2)
34         {
35             float64x2_t vdst = vld1q_f64(dst + i);
36             float64x2_t vsrc = vld1q_f64(src + i);
37             float64x2_t vres = vaddq_f64(vdst, vmulq_f64(valpha, vsrc));
38             vst1q_f64(dst + i, vres);
39         }
40         for (; i < n; ++i) dst[i] += alpha * src[i];
41     #else
42         for (int i = 0; i < n; ++i) dst[i] += alpha * src[i];
43     #endif
44 }

```

其正确性来自于：NEON 主循环仅将原本的逐元素运算改写为每次处理两个连续元素；当 n 为奇数时，尾部循环继续处理剩余元素。因此，该实现与标量循环在数学意义上等价。唯一差别在于浮点加法的累加顺序可能略有变化，但最终测试中的重构误差和正交误差仍保持在可接受范围内。

3.3 Householder 中访存方向重排

原始左 Householder 中计算 $w = v^T B_{sub}$ 的写法如下：

Listing 2: 原始 w 计算方式：按列累加

```

1  std::vector<double> w(n - k, 0.0);
2  for (int j = 0; j < n - k; ++j)
3      for (int i = 0; i < m - k; ++i)
4          w[j] += v[i] * B.at(k + i, k + j);

```

该写法在逻辑上直接，但对行主序矩阵不友好。固定列 j 而遍历行 i 时，相邻访问之间跨越整行长度，缓存局部性较差，也不便于连续向量加载。

优化后将循环重排为按行累加：

Listing 3: 优化后 w 计算方式：按行连续累加

```
1  std::vector<double> w(n - k, 0.0);
2  for (int i = 0; i < m - k; ++i)
3  {
4      const double *brow = &B.at(k + i, k);
5      add_scaled_contiguous(w.data(), brow, v[i], n - k);
6  }
```

该重排仍然计算同一个 $w_j = \sum_i v_i B_{ij}$ ，只是将内层循环变为连续行片段更新。这样既符合矩阵行主序存储，也能复用 `add_scaled_contiguous` 中的 NEON 向量更新。

rank-1 更新也采用连续行更新：

Listing 4: B 子矩阵的连续行更新

```
1  for (int i = 0; i < m - k; ++i)
2  {
3      double *brow = &B.at(k + i, k);
4      add_scaled_contiguous(brow, w.data(), -beta * v[i], n - k);
5  }
```

正交矩阵 U, V 的累计更新同样使用连续点积和连续 scaled-add。由于每次更新都等价于右乘或左乘对应的 Householder 正交变换，所以 $A = UBV^T$ 的分解不变量仍然成立。

3.4 GKH 行旋转指针化与循环展开

GKH 阶段最终采用的优化不是显式 NEON，而是对 `apply_left_rows` 使用行指针访问和 4 路循环展开，如代码 5 所示。

Listing 5: GKH 行旋转的指针访问与 4 路循环展开

```
1  static void apply_left_rows(Matrix &M, int r0, int r1, double c, double s)
2  {
3      int n = M.cols();
4      double* row0 = &M.at(r0, 0);
5      double* row1 = &M.at(r1, 0);
6
7      int j = 0;
8      for (; j + 3 < n; j += 4)
9      {
10         double a0 = row0[j],    b0 = row1[j];
11         double a1 = row0[j + 1], b1 = row1[j + 1];
12         double a2 = row0[j + 2], b2 = row1[j + 2];
13         double a3 = row0[j + 3], b3 = row1[j + 3];
14
15         row0[j]      = c * a0 + s * b0;
```

```

16         row1[j]      = -s * a0 + c * b0;
17         row0[j + 1] = c * a1 + s * b1;
18         row1[j + 1] = -s * a1 + c * b1;
19         row0[j + 2] = c * a2 + s * b2;
20         row1[j + 2] = -s * a2 + c * b2;
21         row0[j + 3] = c * a3 + s * b3;
22         row1[j + 3] = -s * a3 + c * b3;
23     }
24
25     for (; j < n; ++j)
26     {
27         double a = row0[j];
28         double b = row1[j];
29         row0[j] = c * a + s * b;
30         row1[j] = -s * a + c * b;
31     }
32 }

```

该优化有效的原因在于：行旋转访问的是两整行，行主序存储下内存具有连续性；其次，提前取出 `row0`、`row1` 后，内层循环不再反复执行 `r * cols + j` 的下标计算；最后，4 路展开减少循环控制开销，并为编译器指令调度提供更多独立操作。与显式 NEON 相比，该实现引入的额外广播、装载和存储指令更少，因此表现更为稳定。

3.5 未实现并行点的说明

GKH 中的收敛扫描和活动块划分确实具有并行潜力。例如 `split_active_blocks` 会扫描上次对角线元素是否足够小，并按断点划分活动块。但本实验未将其作为核心 SIMD 实现，原因是：其计算量相对矩阵更新很小；逻辑分支较多；活动块并行更接近 OpenMP/Pthread 的多线程任务调度，而非 SIMD 数据通路优化。因此，本实验将这部分作为未来工作，而不是强行加入当前 SIMD 版本。

4 实验环境与测试规范

4.1 实验环境

实验在课程提供的 OpenEuler 服务器集群上完成。相关环境如下：

- 操作系统：OpenEuler 2203 SP4；
- 架构：AArch64 / ARMv8；
- SIMD 指令集：ARM NEON / ASIMD；
- 编译器：g++；
- 默认编译优化等级：-O2；
- 测试命令：`bash test.sh 1 1 1 -O 02 -s 20260409`；
- 固定随机种子：20260409。

4.2 测试规范与数据记录

实验过程中未修改 `main.cpp`、`test.sh` 和 `qsub.sh`。所有正式运行均通过课程脚本提交到计算节点执行。不同版本的 `test.o`、`test.e` 和说明文档统一保存到 `results/` 目录，便于追踪实验过程。

由于服务器为共享环境，运行时间会受到节点负载、调度等待、缓存状态等因素影响。因此，本报告主要关注不同版本之间的趋势，并结合消融实验、编译优化等级和汇编结果进行解释。

5 实验结果与性能分析

5.1 正确性验证

所有核心版本均通过课程测试。测试样例包括固定值 5×5 、随机 8×8 、近秩亏损 10×8 、随机 10×8 和随机 1000×1000 矩阵。验证指标包括重构误差 $\|A - USV^T\|_F$ 、相对重构误差、正交误差 $\|U^T U - I\|_F$ 和 $\|V^T V - I\|_F$ 、对角结构误差、奇异值降序性与非负性。

以 final 版本为例， 1000×1000 样例中重构误差为 1.86375×10^{-10} ，相对重构误差为 3.22033×10^{-13} ，正交误差保持在 10^{-13} 量级，最终 5 / 5 PASS。这说明 SIMD 优化虽然改变了部分浮点累加顺序，但没有破坏 SVD 分解正确性。

5.2 递进式消融实验

为分析不同优化的独立贡献，本实验设计了五个版本：`baseline`、`gkh_neon_attempt`、`gkh_left_unroll`、`bidiag_only` 和 `final`。结果如表 1 所示。这里选择表格呈现而非截图，原因是数值对比是主要信息，表格更便于计算加速比和进行横向比较；原始日志已保存在 `results/` 目录中，可作为附加证据。

表 1: 递进式优化消融实验结果（单位：ms）

版本	Bidiagonalization	GKH	Total	说明
baseline	10342.50	41470.20	51812.70	原始版本
gkh_neon_attempt	7225.14	35229.10	42454.24	显式 NEON 尝试
gkh_left_unroll	8756.41	25197.00	33953.41	仅优化 GKH 行旋转
bidiag_only	3716.30	42036.20	45752.50	仅优化 Householder
final	3651.82	30509.90	34161.72	综合优化

需要说明的是，表中各版本并非在完全隔离的硬件环境下连续重复测得，服务器节点负载和调度状态会带来一定波动。因此，消融实验主要关注“被修改阶段”的耗时变化；未修改阶段的耗时仅作为运行环境下的参考值，不单独用于归因。

从表中可以看出，Householder 优化显著降低了上二对角化阶段耗时。`bidiag_only` 版本将该阶段从 10342.50 ms 降至 3716.30 ms，加速比约为 2.78 $\times$ 。这说明访存方向重排和连续 NEON 更新非常适合 Householder 中的大规模矩阵-向量和 rank-1 更新。

GKH 阶段的情况相对复杂。显式 NEON 尝试虽然在部分运行中有收益，但早期多次测试表明其稳定性不足。最终采用的 `gkh_left_unroll` 版本将 GKH 耗时降至 25197.00 ms，说明对行旋转进行指针化和循环展开相比直接为行列旋转引入显式 NEON 更为稳妥。

final 版本总耗时为 34161.72 ms，相比 baseline 的 51812.70 ms，整体加速比约为：

$$\frac{51812.70}{34161.72} \approx 1.52. \quad (10)$$

5.3 编译优化等级对比

为分析手写 SIMD 与编译器优化的关系，本实验在 final 版本上测试了 O0、O1、O2、O3 和 Ofast。结果如表 2 所示。

表 2: 不同编译优化等级下 final 版本性能（单位：ms）

优化等级	通过情况	Bidiagonalization	GKH	Total	备注
O0	5 / 5	25341.80	127001.00	152342.80	无编译优化，显著较慢
O1	5 / 5	3183.51	32118.00	35301.51	基础优化后大幅提升
O2	5 / 5	3011.46	25938.50	28949.96	默认优化等级，较均衡
O3	5 / 5	3221.97	28280.70	31502.67	未明显优于 O2
Ofast	5 / 5	3268.40	22045.70	25314.10	最快但需关注浮点语义

O0 下总耗时达到 152342.80 ms，说明本程序中大量循环和函数调用对编译器优化高度敏感。从 O0 到 O1，总耗时大幅下降至 35301.51 ms，表明基本内联、寄存器分配和循环优化已经显著改善性能。O2 是课程默认使用的优化等级，整体表现稳健。O3 未进一步优于 O2，说明更激进的优化并不总能改善此类数值迭代程序。Ofast 在本组实验中总耗时最低，但会放宽部分浮点语义，因此只作为性能参考。

5.4 perf 端到端观察

实验中尝试使用 `perf stat` 进行硬件计数器分析。登录节点存在 `/usr/bin/perf`，但 `qsub` 分配到的计算节点未安装 `perf`，且无法指定登录节点作为计算节点。因此，本实验采用如下端到端命令：

Listing 6: `perf` 包裹 `test.sh` 的端到端观察命令

```
1 perf stat bash test.sh 1 1 1 -O 02 -s 20260409
```

需要说明的是，该命令统计的是登录节点上 `test.sh`、编译、`qsub` 提交、文件传输、输出收集和等待过程，而不是计算节点上 `main` 进程的纯硬件事件。因此，表 3 中的 `cycles`、`instructions` 和 `branch-misses` 不能直接等同于 SVD 主程序内部的硬件计数器结果，只能作为端到端运行流程的辅助观察。

表 3: `perf stat` 包裹 `test.sh` 的端到端结果

版本	task-clock(ms)	cycles	instructions	IPC	branch-misses	elapsed(s)	user(s)	sys(s)
baseline	5427.84	12701362613	10720045368	0.84	115655385	65.4436	5.0041	0.4144
gkh_left_unroll	5393.99	12610208812	10764576353	0.85	116554868	45.3987	4.9556	0.4199
bidiag_only	5750.92	13532306885	11903096822	0.88	127715086	60.7548	5.3558	0.3856
final	5932.59	13916540358	11948456848	0.86	127711446	40.9400	5.4301	0.4864

从表中可以看到，各版本的 `task-clock` 均在 5–6 秒量级，而 `elapsed` 时间却达到 40–65 秒。这说明端到端运行中存在大量等待时间，主要来自 `qsub` 调度、远程节点执行、文件传输和输出回收，而不是登录节点本身持续执行计算。因此，`elapsed` 更能反映课程脚本整体运行成本，`task-clock` 则主要反映登录节点上脚本、编译和调度相关命令消耗的 CPU 时间。

此外，各版本的 IPC 大约在 0.84–0.88 之间，说明登录节点统计到的这部分流程并不是单纯的高密度浮点计算，而混合了 shell 脚本、编译、I/O 和等待逻辑。`final` 版本的端到端 `elapsed` 最低，为 40.9400 s，与程序内部计时中 `final` 版本整体更快的趋势一致；但 `final` 的 `cycles` 和 `instructions` 并没有最低，这再次说明该 `perf` 结果不是对计算节点 `main` 的纯采样结果，不能用来精确判断 SVD 算子的 `cache miss` 或 IPC 改善。

因此，本实验对 `perf` 数据采取保守解释：它不能替代对计算节点主程序的直接采样，但仍能反映课程提交脚本的端到端运行特征，并说明本实验的总等待时间并不完全由 SVD 计算本身决定。最终性能结论仍主要依据程序内部输出的 `time bidiagonalization(ms)` 和 `time gkh iteration(ms)`。若要获得严格的硬件事件分析，需要在实际计算节点上安装或开放 `perf`，并直接执行 `perf stat ./main 20260409`。

5.5 汇编级分析

为了验证 NEON 辅助函数是否真正被编译为底层向量指令，本实验使用 `g++ -O2 -march=native -S` 生成汇编文件，并 `grep fmla, fmul, fadd, ldr q, str q` 等浮点或向量相关指令。统计结果如表 4 所示。

表 4: 汇编中 `grep` 匹配到的浮点/向量相关指令行数

文件	匹配行数
<code>bidiagonalization_baseline_vector_instr.txt</code>	8
<code>bidiagonalization_final_vector_instr.txt</code>	15
<code>gkh_baseline_vector_instr.txt</code>	15
<code>gkh_final_vector_instr.txt</code>	23

`baseline` 的 `bidiagonalization.cpp` 中主要出现标量双精度浮点指令：

Listing 7: `baseline` 中的标量浮点指令示例

```
1 fadd    d0, d8, d0
2 fmul    d1, d8, d1
```

其中 `d` 寄存器表示 64-bit 双精度标量寄存器。这说明原始版本中的许多计算仍以标量浮点操作为主，虽然编译器可能会进行一定的局部优化，但代码结构中的跨行访问和 `Matrix::at()` 下标计算限制了自动向量化空间。

`final` 版本的 `bidiagonalization.cpp` 中则出现了明显的 NEON 向量指令：

Listing 8: `final` 中 Householder 优化对应的 NEON 向量指令

```
1 ldr     q1, [x4], 16
2 ldr     q2, [x5], 16
3 fmla    v0.2d, v2.2d, v1.2d
4
5 ldr     q2, [x3], 16
```

```

6  ldr    q1, [x2]
7  fmla   v1.2d, v3.2d, v2.2d
8  str    q1, [x2], 16

```

其中 `q` 寄存器表示 128-bit 向量寄存器, `v?.2d` 表示一个 NEON 向量中包含两个 double 元素。`ldr q` 和 `str q` 对应 128-bit 向量加载和存储, `fmla v?.2d` 对应双精度向量 fused multiply-add。该结果表明 `dot_product_contiguous` 和 `add_scaled_contiguous` 中的连续数组计算已被编译为 NEON 向量运算, 说明 Householder 优化不仅体现在源代码层面的 Intrinsics 调用, 也落实到了 AArch64/NEON 指令路径上。

从算法角度看, 这与前面的优化设计一致。Householder 阶段的 $w = v^T B_{sub}$ 和 $B_{sub} \leftarrow B_{sub} - \beta v w^T$ 本质上是连续点积和连续向量更新; 优化后内层访问的是连续行片段, 能够自然映射为 `ldr q-fmla-str q` 的向量化执行流。因此, 汇编结果为上二对角化阶段的加速提供了底层证据。

相比之下, GKH final 版本虽然匹配到的浮点相关指令行数增加, 但主要仍是 `fmul d...` 和 `fadd d...` 这类标量双精度指令。这与实现策略一致: GKH 阶段最终采用的是行指针访问和 4 路循环展开, 而不是显式 NEON Intrinsics。也就是说, GKH 的性能收益主要来自减少 `Matrix::at()` 带来的重复地址计算、改善行访问连续性、降低循环控制开销, 并为编译器指令调度提供更多独立标量操作, 而不应被解释为显式 NEON 向量化收益。

还需要注意, 汇编中匹配到的指令行数并不等同于性能高低。指令行数增加可能表示更多向量化操作, 也可能只是代码展开或编译器生成路径变多。因此, 本实验使用汇编分析的目的不是简单比较“谁的指令更多”, 而是确认关键优化路径是否生成了预期的 NEON 指令。结合消融实验结果可以看出: Householder 阶段既有源码层面的连续访存重排, 又有汇编层面的 NEON 指令证据, 因此其加速原因较为明确; GKH 阶段则主要依赖轻量级结构优化, 显式 NEON 尝试虽然被实现和测试, 但未作为最终方案。

6 进阶探索与反思

6.1 手写 SIMD 与编译器自动优化

本实验的结论是: 手写 SIMD 并不是简单地将标量循环替换为 Intrinsics, 而是要先判断数据布局、访存模式和循环结构是否适合向量化。对于 Householder 上二对角化阶段, 原始代码虽然数学表达清晰, 但存在按列访问行主序矩阵的问题。以 $w = v^T B_{sub}$ 为例, 原始写法固定列下标、遍历行下标, 相邻访问之间跨越整行长度, 缓存局部性较差, 编译器也难以自动将其重排为稳定的连续向量 load/store。因此, 单纯依赖 `-O2` 或 `-O3` 很难获得理想的自动向量化效果。

本实验对 Householder 的优化首先改变的是计算组织方式, 而不是直接套用指令。通过将按列累加改写为按行连续累加, 内层循环变为对连续数组片段执行 scaled-add; 随后再使用 `float64x2_t`、`vld1q_f64`、`vst1q_f64` 等 NEON Intrinsics 进行双精度向量化。换言之, 优化真正生效的原因是“数学等价变换 + 连续访存 + SIMD 指令映射”三者共同作用。如果没有前面的访存重排, 即使强行写 NEON, 也可能因为数据不连续而无法取得稳定收益。

相比之下, GKH 阶段体现了另一个结论: 并不是所有浮点循环都适合显式 SIMD。GKH 的 bulge chasing 具有较强的顺序依赖, 一次旋转会改变后续旋转需要消去的元素; 同时, 列旋转在行主序矩阵中存在跨行访问, 局部性不如行旋转。实验中的 `gkh_neon_attempt` 版本说明, 显式 NEON 在这类小粒度、访存不完全连续的场景中并不一定稳定快于标量实现。最终采用的行指针访问与 4 路循环展开, 本质上是一种更轻量、更贴合数据布局的优化: 它减少了 `Matrix::at()` 带来的重复下标计算, 降低了循环控制开销, 同时没有引入过多向量装载和尾部处理成本。

编译优化等级实验也印证了这一点。O0 到 O1 的性能提升非常明显，说明编译器的基础优化、内联和寄存器分配对该程序十分重要；但 O3 不一定优于 O2，说明更激进的自动优化并不必然适合数值迭代程序。Ofast 虽然在本实验中耗时最低，但其可能放宽浮点语义，因此更适合作为性能参考，而不是默认正确性优先的实验配置。综合来看，合理的 SIMD 优化应当与编译器优化互补：程序员负责暴露连续访存和规则计算结构，编译器负责完成底层调度、寄存器分配和部分指令选择。

6.2 关于收敛扫描与活动块并行的反思

助教讲解中提到的收敛元素扫描和活动块分块并行，是 GKH 迭代中很自然的并行化方向。本实验没有将其实现为最终 SIMD 代码，并不是忽略该方向，而是基于本次实验范围和计算占比作出的取舍。

收敛扫描主要遍历上次对角线元素 e_i ，判断其是否满足阈值条件并将其置零。这类操作属于低计算密度循环，主要由绝对值、比较和分支构成。即使使用 SIMD 同时比较多个元素，其收益也可能被分支处理、块边界构造和后续逻辑开销抵消。更重要的是，在本框架中真正耗时的是 Householder 阶段的大规模矩阵块更新，以及 GKH 阶段对 B, U, V 的旋转更新。相比之下，扫描本身的时间占比相对较小，因此不应作为第一优先级的 SIMD 优化目标。

活动块分块并行更适合多线程或多进程模型。当若干上次对角元素收敛为 0 后，矩阵可以拆分为多个互不耦合的子问题，这些子问题理论上可以分配给不同线程独立迭代。但这属于任务级并行，而不是典型 SIMD 数据级并行；并且在随机 1000×1000 样例中，GKH 前期往往仍以较大的活动块为主，块内 bulge chasing 的顺序依赖较强。因此，本实验将该方向作为未来 OpenMP/Pthread 扩展，而不是混入当前单核 SIMD 优化中。这样的取舍也使实验目标更清晰：本实验重点验证数据级 SIMD 对 Householder 和局部 Givens 更新的影响，而不是同时引入多线程调度带来的额外变量。

6.3 AI 辅助实验设计与验证反思

本实验过程中使用 AI 辅助进行架构判断、代码优化思路讨论、性能异常分析和报告结构整理。AI 的主要价值不在于直接生成最终代码，而在于帮助梳理可能的优化路径，并在实验结果异常时提供解释角度。

例如，在实验初期，曾考虑使用 `immintrin.h` 一类 x86 SIMD Intrinsics，但服务器环境实际为 AArch64，因此应使用 ARM NEON/ASIMD。AI 在这一阶段帮助明确了平台和指令集之间的对应关系，避免了在错误架构上继续尝试。之后，在分析 Householder 优化时，AI 帮助将 $w = v^T B_{sub}$ 的计算从数学公式拆解为连续点积和 scaled-add，从而形成可复用的 NEON 辅助函数。该过程并非简单地由 AI 生成代码，而是借助 AI 将算法结构、数据布局和代码实现对应起来。

在 GKH 优化阶段，AI 的作用更多体现在反例分析上。显式 NEON Givens 旋转并未稳定带来收益，部分运行结果甚至变慢。AI 提供的解释包括：double NEON 向量宽度有限、列旋转访存不连续、bulge chasing 粒度较小、编译器对简单标量循环已有优化等。后续实验结果也支持这一判断，因此最终版本选择了行指针访问与循环展开，而不是为了增加“SIMD 外观”而保留显式 NEON。

同时，本实验也说明 AI 建议必须经过实际测试验证。不同服务器节点的运行时间存在明显波动，单次运行结果不能直接作为最终结论；代码优化也可能因为编译器、数据规模和缓存状态不同而表现不稳定。因此，最终报告中保留了 baseline、GKH NEON 尝试、GKH 行展开、bidiag-only、final、编译优化等级、perf 和汇编分析等多组证据，用实验数据而不是单次经验来支撑结论。总体来说，AI 更适合作为实验设计和问题排查的辅助工具，而不能替代实际编译、运行、保存日志和消融分析。

7 结论

本实验围绕 SVD 分解中的 Householder 上二对角化和 GKH 迭代两个阶段进行了 SIMD 优化，并得到以下结论：

- 1) Householder 变换是本实验最适合 SIMD 优化的核心阶段。通过将按列访问重排为按行连续访问，并使用 ARM NEON 辅助函数优化连续点积和 scaled-add，上二对角化阶段取得显著加速。
- 2) GKH 阶段不适合盲目套用显式 NEON。最终采用的 `apply_left_rows` 指针化与 4 路循环展开更加稳定，能够减少 `Matrix::at()` 下标计算和循环控制开销。
- 3) 消融实验表明，final 版本同时降低了上二对角化与 GKH 两个阶段耗时。在代表性 O2 实验中，总耗时由 baseline 的 51812.70 ms 降至 final 的 34161.72 ms，整体加速比约为 1.52 倍。
- 4) 编译优化等级对性能影响显著。O0 下程序极慢，O1 后性能大幅改善，O2 是较稳妥的默认配置，Ofast 虽然最快但需要关注浮点语义变化。
- 5) 汇编级分析验证了 final 版本中 Householder 优化确实生成了 `fmla v?.2d, ldr q, str q` 等 AArch64/NEON 双精度向量指令。

受限于本次实验主题和框架要求，收敛扫描并行和活动块多线程调度没有在最终代码中实现。后续可以进一步探索 OpenMP 多线程并行活动块处理、跨节点 MPI 分布式矩阵分解，或将 SVD 算子迁移到 GPU/Triton 环境中进行更大规模的算子优化。

A 代码开放获取

GitHub 项目仓库：<https://github.com/ZR-1N/Parallel-Course-Experiments>

实验分支：`svd-simd`

B 实验结果文件组织

Listing 9: 实验结果目录组织

```

1 results/
2   00_baseline/
3   01_gkh_neon_attempt/
4   02_gkh_left_unroll/
5   03_bidiag_only/
6   04_final/
7   05_opt_level/
8   06_perf_testsh/
9   07_asm/

```

其中,00_baseline 到 04_final 用于消融实验,05_opt_level 用于编译优化等级对比,06_perf_testsh 记录端到端 perf 观察, 07_asm 保存汇编级分析文件。

C AI 辅助编程对话摘录

C.1 架构选择

问题：OpenEuler 服务器上应该使用哪种 SIMD 指令集？

回答摘要：服务器为 AArch64/ARMv8 架构，应使用 ARM NEON/ASIMD，而不是 x86 AVX。由于本实验使用 double，NEON 128-bit 向量一次可处理两个 double 元素，对应 float64x2_t。

C.2 GKH 显式 NEON 性能不稳定

问题：为什么将 Givens 旋转直接改为 NEON 后不一定更快？

回答摘要：GKH 旋转的计算粒度较小，且 `apply_right_cols` 存在跨行访问，显式 NEON 的 load/store、广播和 lane 操作开销可能抵消计算收益；同时 -O2 下编译器已经能较好优化简单标量循环。因此，最终选择行指针访问与循环展开。

C.3 Householder 访存重排

问题：如何优化 Householder 中的 $w = v^T B_{sub}$ ？

回答摘要：原始写法按列访问矩阵，不符合行主序存储。将循环重排为按行累加 `w += v[i] * B_row` 后，内层访问连续内存，可以调用 NEON 辅助函数进行连续 scaled-add，同时保持数学等价。

C.4 perf 与汇编分析

问题：如何在 qsub 环境下进行性能分析？

回答摘要：由于计算节点未安装 perf，无法直接对计算节点上的 main 进行硬件事件采样。可以使用 `perf stat bash test.sh ...` 进行端到端辅助观察，但核心性能结论应以程序内部计时为主。汇编分析则可以通过 `g++ -S` 静态生成，验证 `fmla v?.2d` 等 NEON 指令是否出现。

参考文献

- [1] Gene H. Golub and William Kahan. Calculating the singular values and pseudo-inverse of a matrix. *Journal of the Society for Industrial and Applied Mathematics, Series B: Numerical Analysis*, 2(2):205–224, 1965.
- [2] Gene H. Golub and Christian Reinsch. Singular value decomposition and least squares solutions. *Numerische Mathematik*, 14:403–420, 1970.
- [3] Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. Society for Industrial and Applied Mathematics, Philadelphia, PA, 2 edition, 2002.
- [4] Arm Ltd. Arm neon intrinsics reference. Arm Developer Documentation, 2024. Available online: <https://developer.arm.com/architectures/instruction-sets/intrinsics/>.